Servidor concurrente mediante procesos

Tomás Fernández Pena
José Carlos Cabaleiro Domínguez

Grado en Ingeniería Informática

Universidade de Santiago de Compostela

Redes, 2º Curso GrEI

citius.usc.es

Índice

1 Funciones de gestión de procesos

2 Servidor concurrente usando procesos

Índice

1 Funciones de gestión de procesos

2 Servidor concurrente usando procesos

Función `fork`: creación de procesos

```
pid_t fork(void)
```

- Crea un nuevo proceso
- Después de la llamada habrá dos procesos, padre e hijo
  - Continúan ejecutándose a partir del punto en el que se realizó la llamada con los valores de las variables previos
  - Cada uno con sus variables independientes
- Valor devuelto: al proceso hijo un 0, al padre el pid del hijo y -1 en caso de error
- Se necesitan las cabeceras `unistd.h` y `sys/types.h`

Función `waitpid`: espera por un proceso hijo

```
pid_t waitpid(pid_t pid, int *status, int options)
```

- Esperar a que el proceso hijo cambie de estado
- Por defecto, espera a que un proceso hijo termine
- Parámetros:
  - `pid` si igual a -1 (o `WAIT_ANY`) proporciona información de cualquier proceso hijo (del primero que termine)
    - Si se desea obtener información de otro hijo, hay que volver a llamar a la función
  - `status` puntero al entero que almacena el estado
  - `options` se puede poner 0 o a `WNOHANG` para que el padre no espere
- Valor devuelto: el número del proceso hijo en caso de éxito y -1 en caso de error
  - 0 en el caso de que no ningún proceso haya terminado (con la opción `WNOHANG`)
- Se necesitan las cabeceras `sys/types.h` y `sys/wait.h`

Esperar a la terminación de los hijos

- El proceso padre debe esperar a la terminación de los hijos
  - Si un proceso hijo termina sin ser esperado por el padre, queda en estado **zombie**
    - Permanece en la tabla de procesos con su valor de retorno
  - La función `waitpid()` usada directamente puede bloquear al padre
  - Se puede usar en una función que maneja la señal `SIGCHLD`
    - Señal enviada al padre cuando un hijo termina
  - Se usa `signal(SIGCHLD, manejador)`
    - Informa al SO que debe usar la función `manejador()` cada vez que recibe una señal del hijo
  - La función `manejador()` se ejecuta cada vez que termina un hijo

```
void manejador(int senial){
      while(waitpid(WAIT_ANY, NULL, WNOHANG)>0);
 }
```

  - De esta forma los hijos que vayan terminando no quedan en estado **zombie**

Índice

1 Funciones de gestión de procesos

2 Servidor concurrente usando procesos

Pasos para hacer un servidor concurrente con procesos

1. Partir del servidor de mayúsculas de la práctica de TCP
2. Al principio llamar a la función `signal()` como se indicó
3. Después de la función `accept()`, llamar a la función `fork()`
4. Construir un `if-else` distinguiendo entre el padre y el hijo
   - Si es el hijo
     - Cerrar el socket de servidor
     - Mostrar en pantalla el número de proceso
     - Implementar el lazo de recepciones y envíos del servidor de partida[1]
     - Cerrar el socket de conexión y salir con `exit(EXIT_SUCCESS)`
   - Si es el padre
     - Cerrar el socket de conexión (no lo necesita)
     - Hacer que continúe en el lazo de atención de múltiples clientes[2]
     - Cerrar el socket de servidor al final como siempre
5. Comprobar que puede atender múltiples clientes **simultáneamente**

[1] Terminará cuando el cliente corte la conexión

[2] Volverá a la función `accept()`